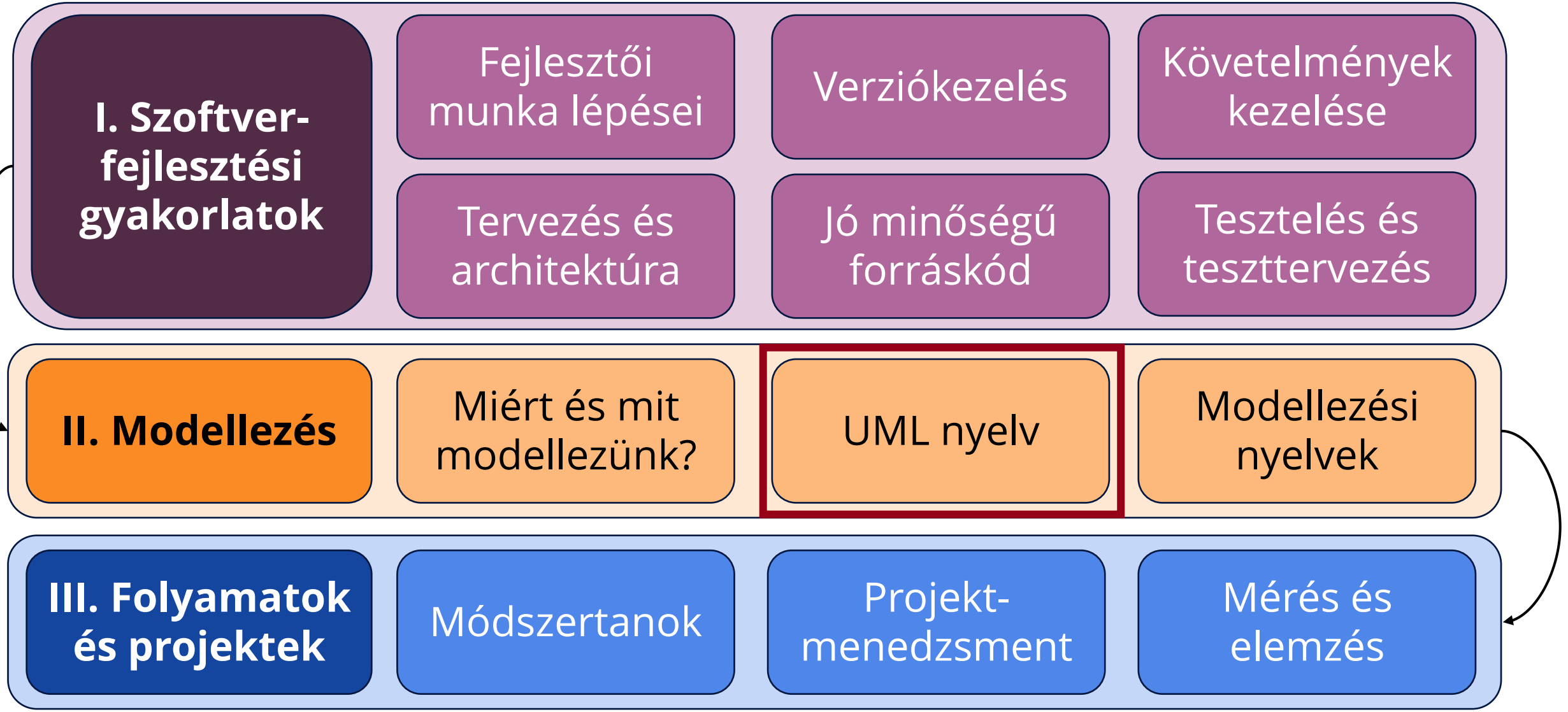
UML struktúra modellek

Dr. Micskei Zoltán, Dr. Gönczy László
https://ftsrg.mit.bme.hu

(Dr. Simon Balázs [IIT] fóliáit is felhasználva)

MŰEGYETEM 1782
BME VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
Mesterséges Intelligencia és Rendszertervezés Tanszék
ftsrg Critical Systems Research Group

Szoftvertechnológia tantárgy témái

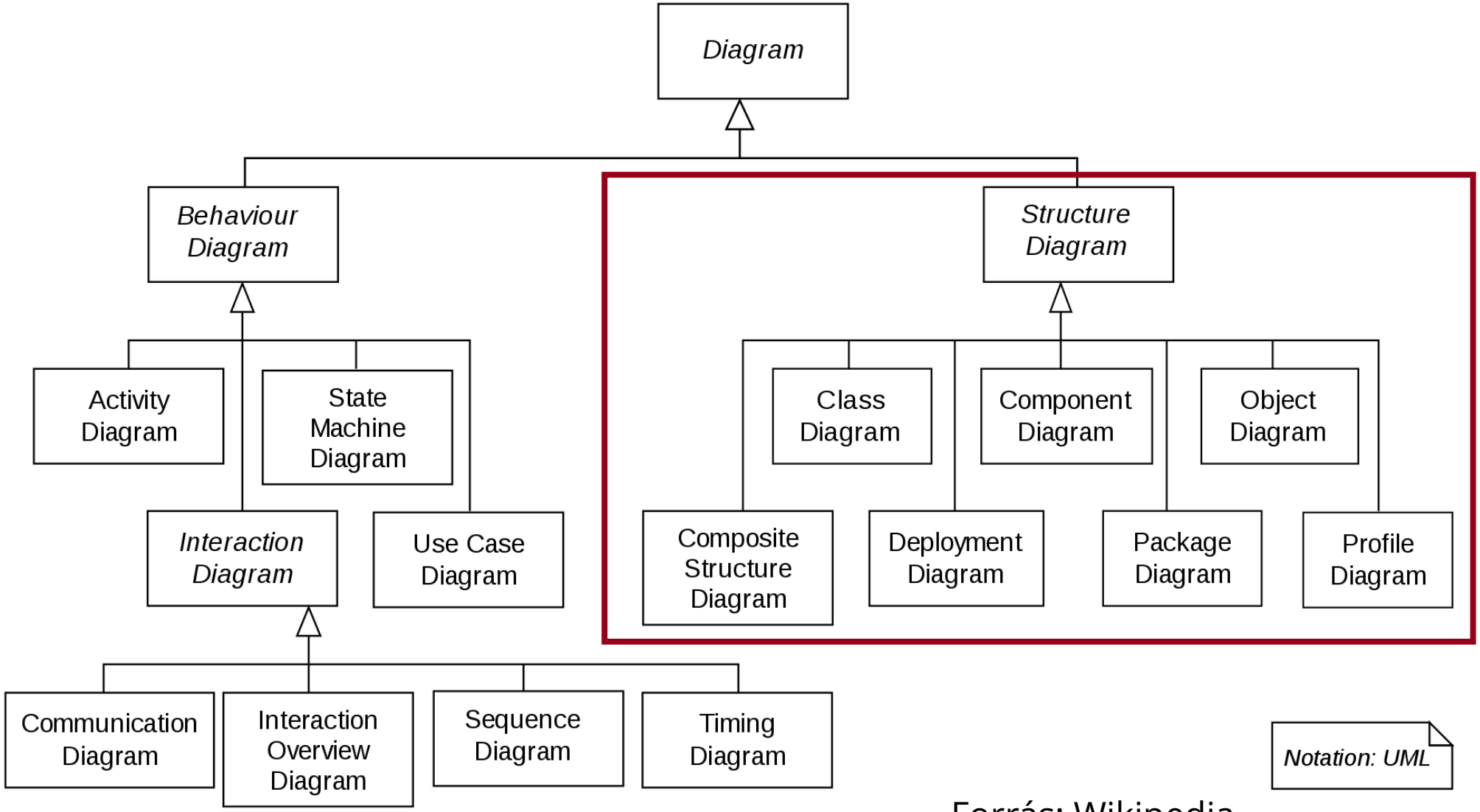


Tanulási eredmények

A tananyag elsajátítása után a hallgató képes lesz

- (K1) felidézni a strukturális modellezés céljait,
- (K3) alkalmazni az UML nyelvet komponensek modellezésére,
- (K3) alkalmazni az UML nyelvet fogalmi és OO modellezésre.

UML diagram típusok



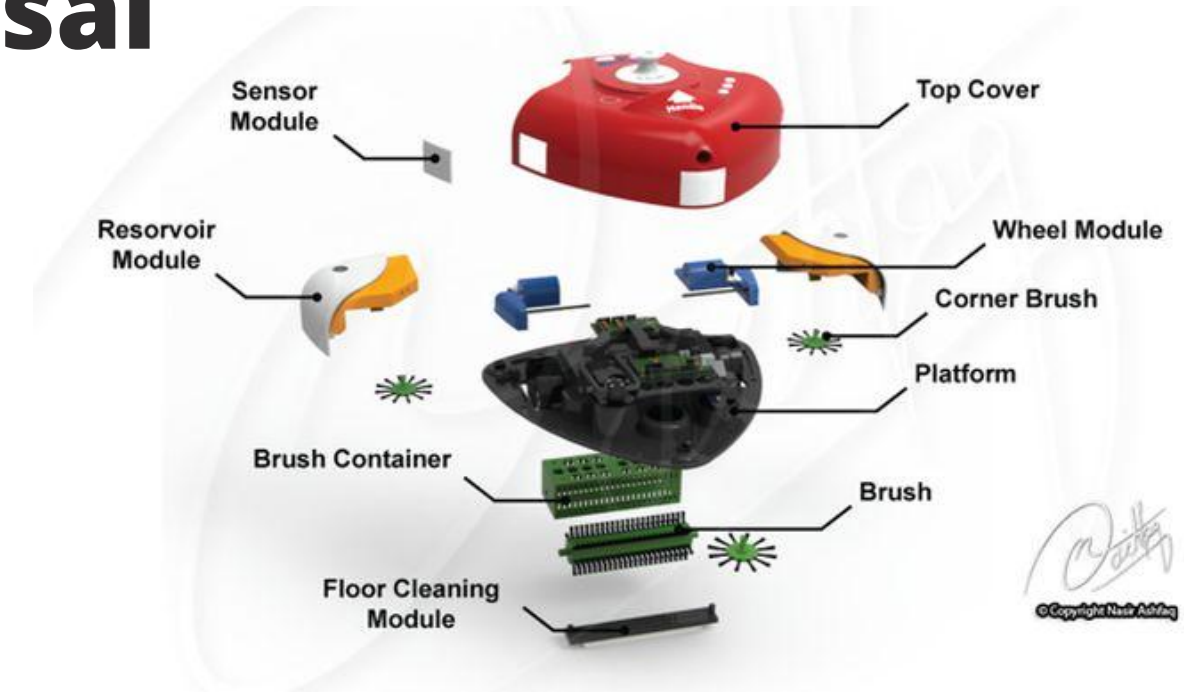
Forrás: Wikipedia

Strukturális modellezés célja

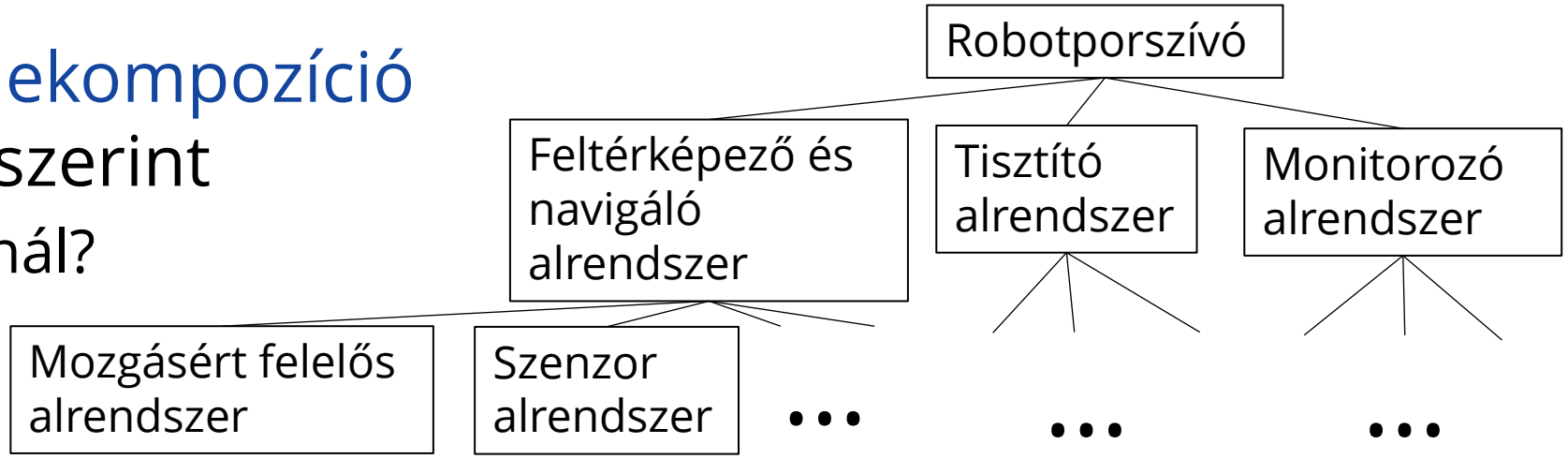
- Rendszer részekre bontása („dekompozíció”)
  - Kisebb egységeket könnyebb megtervezni
  - Részegységek felhasználása
  - Általános célú komponensek használata
- Létező rendszer dokumentálása
  - „Rendszertérkép”
- Adatszerkezet megalkotása
  - Milyen információt kezelünk?

Dekompozíció típusai

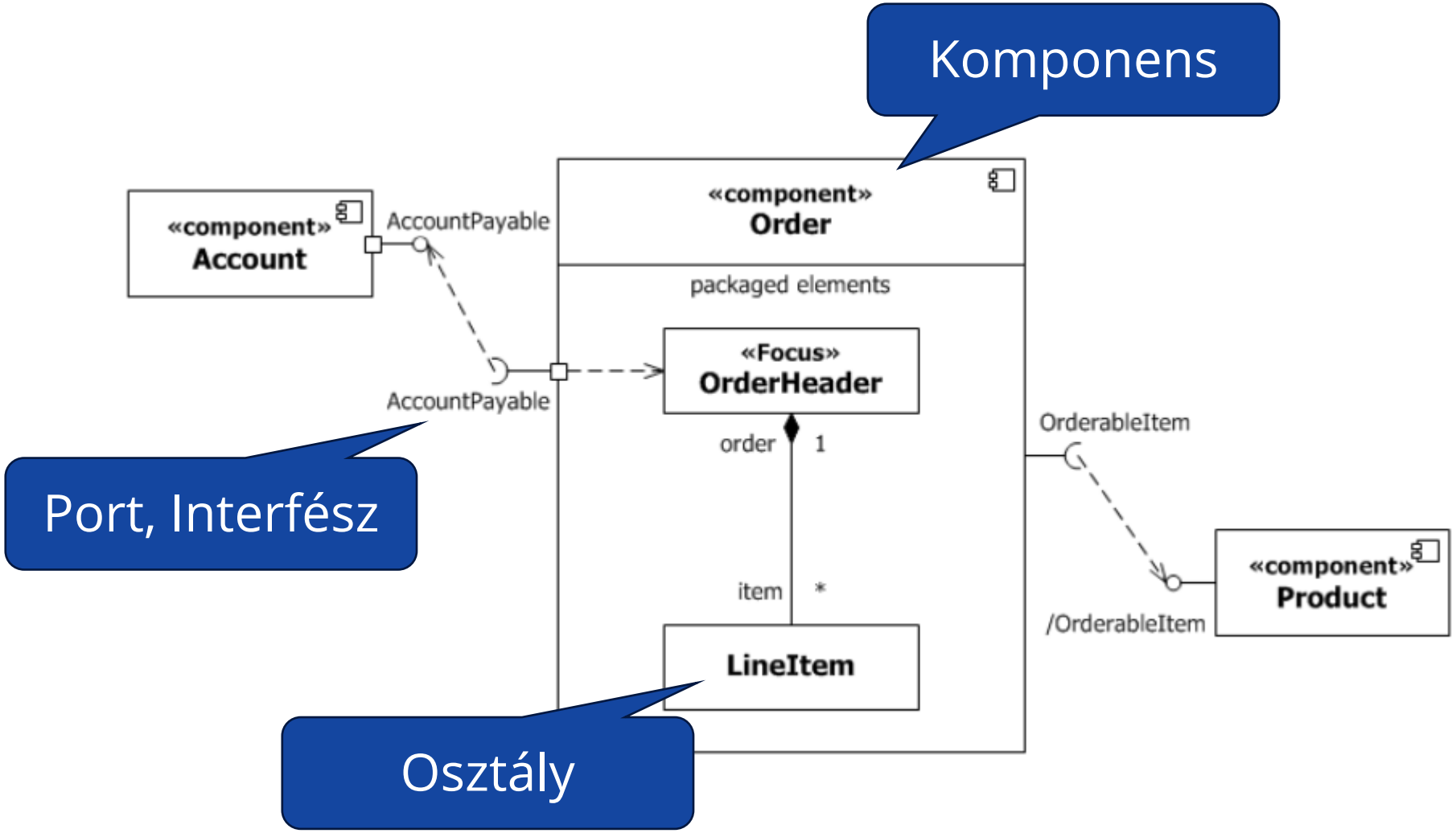
- Fizikai dekompozíció
  - Milyen alkatrész?



- Logikai dekompozíció *funkciók* szerint
  - Mit csinál?



UML struktúra modellek



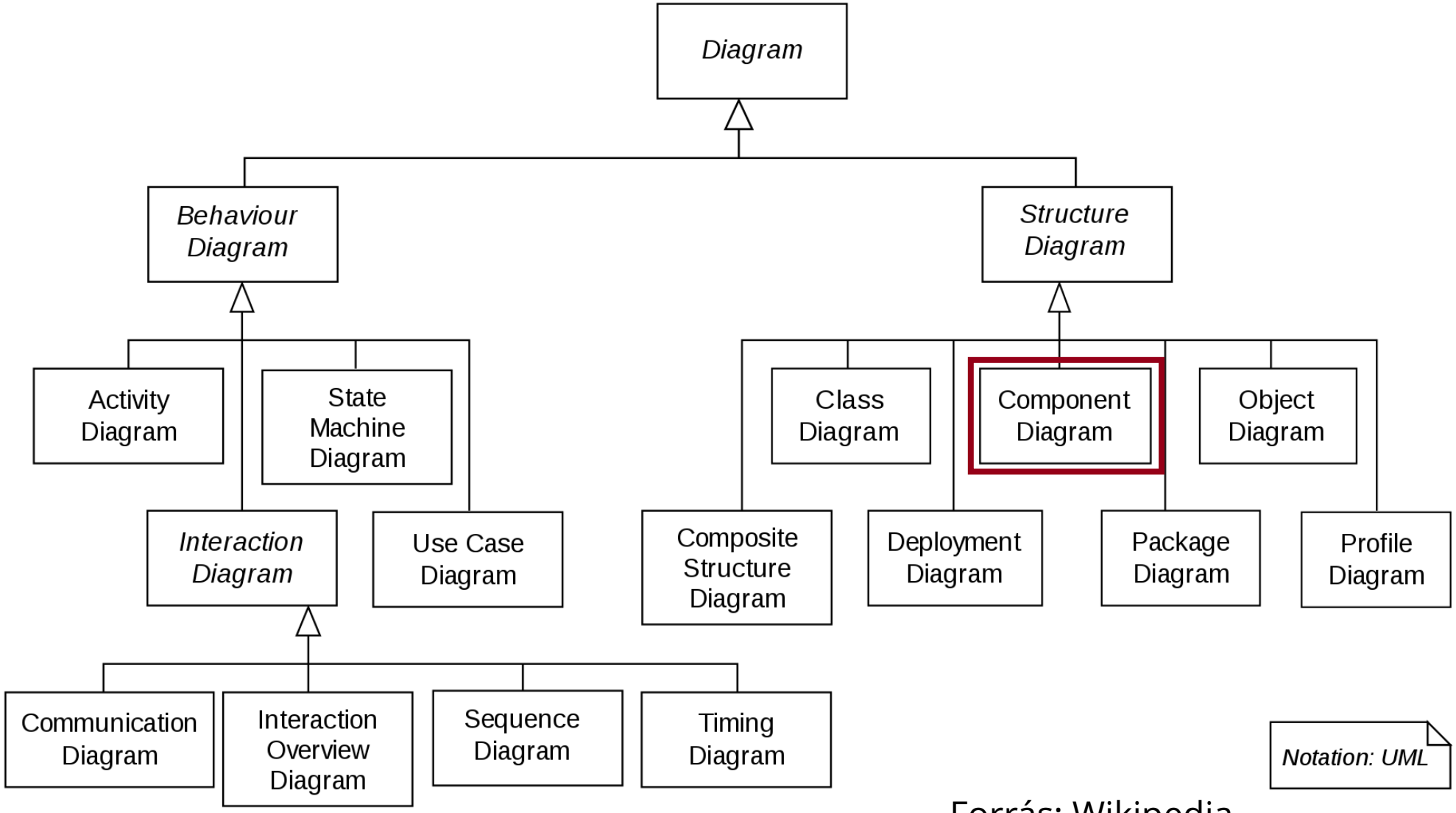
Forrás: UML v2.5.1

UML komponens modellek

Komponens, port, interfész



UML diagram típusok



Forrás: Wikipedia

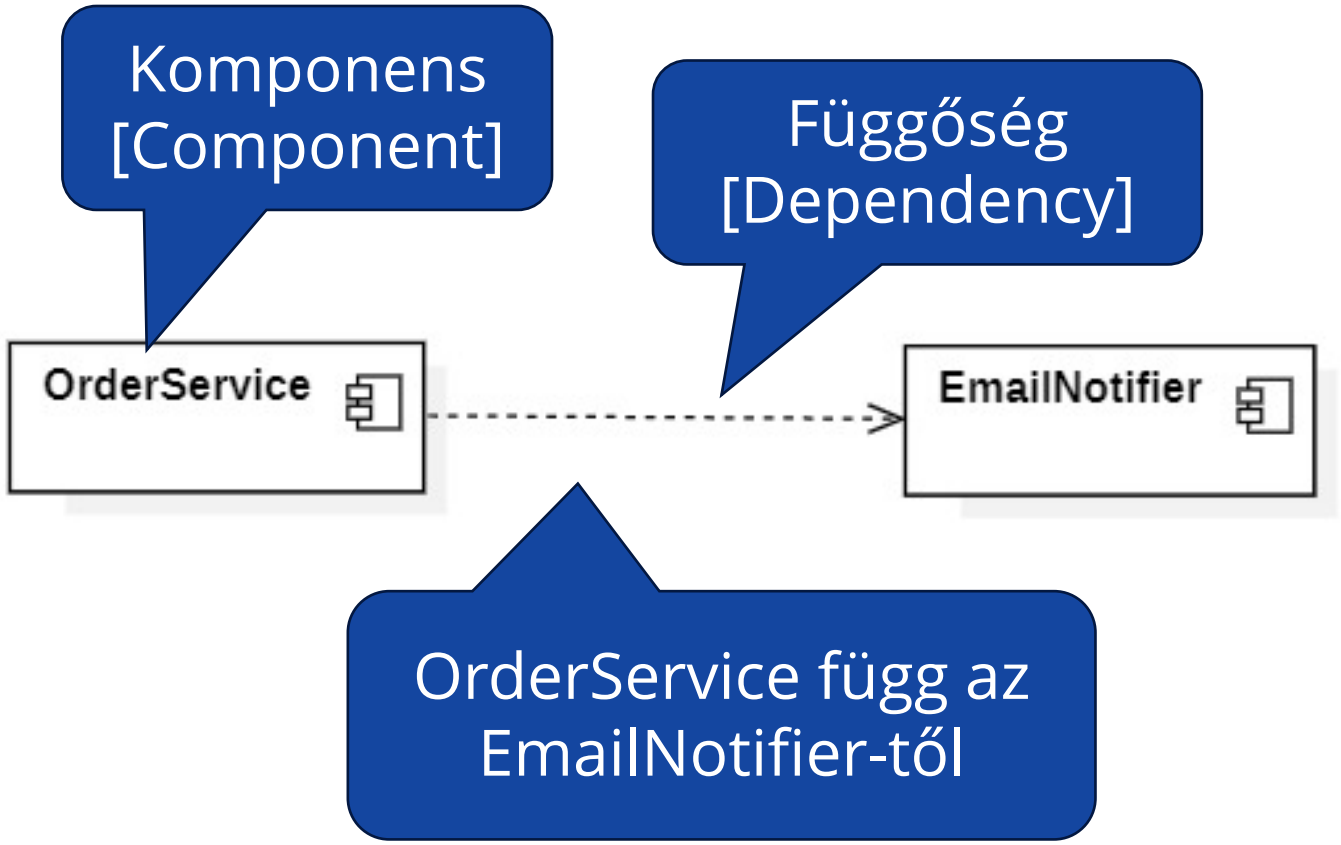
Mi a komponens?

Mechanikai, SW, HW…

> “A komponens a rendszer egy moduláris részét képviseli, ami egységbe zárja a tartalmát, és aminek a megnyilvánulása lecserélhető a környezetében.”
>
> UML Specification 2.5.1

> “A szoftverkomponens a kompozíció egysége, ami szerződésként megadott interfészekkel és csak explicit módon függ a környezetétől. A komponenst önállóan lehet telepíteni, és komponálható más komponensekkel.
>
> Forrás: C. Szyperski. Component Software - Beyond Object-Oriented Programming, 1999

Komponensek és kapcsolatok



- Függőség: szállító [supplier] változása kihathat a kliens [client] viselkedésére
- (Nyíl felőli vége a szállító)
- Pontos függőség és hatás nincs megadva (még)

Motivációs példa

```java
public class OrderService {
  public void placeOrder(Order order) {
    // ... Business logic (e.g. save order)
    EmailNotifier notifier = new EmailNotifier();
    notifier.send(order.getMessage());
  }
}

public class EmailNotifier {
    public void send(Message m) {
        // send email about message
    }
}
```

Mi a baj ezzel a megvalósítással?

```java
public interface Notifier {
  void send(Message m);
}

public class EmailNotifier implements Notifier {
  @Override
  public void send(Message m) {
    // send email about message
  }
}

public class PushNotifier implements Notifier {…}

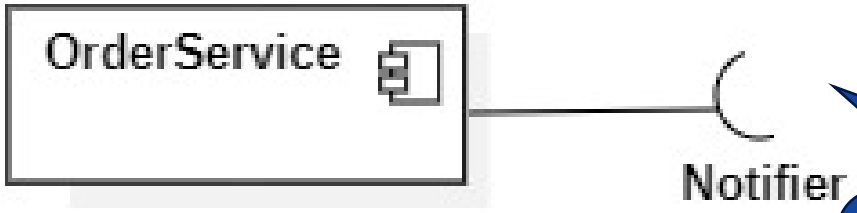
public class OrderService {
  private final Notifier notifier;

  public OrderService(Notifier notifier) {
      this.notifier = notifier;
  }

  public void placeOrder(Order order) {
    // ... Business logic (e.g. save order)
    notifier.send(order.getMessage());
    }
}
```

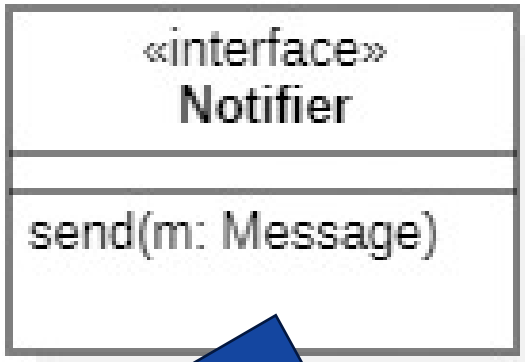
Lazán csatolt megoldás (függőség könnyen cserélhető)

Komponens definiálása interfészekkel

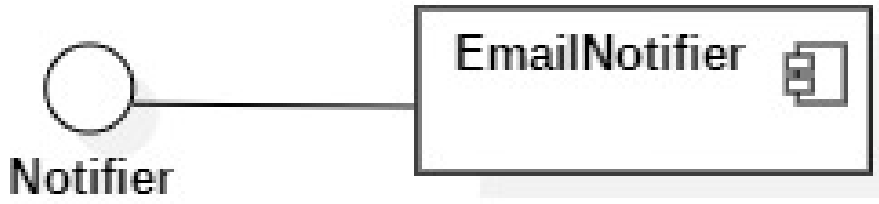


elvárt interfész
[required interface]
„foglalat” [socket]

Interfész: nyilvános, összetartozó „szerződés”



Műveletek, viselkedések…



biztosított interfész
[provided interface]
„nyalóka” [lollipop]

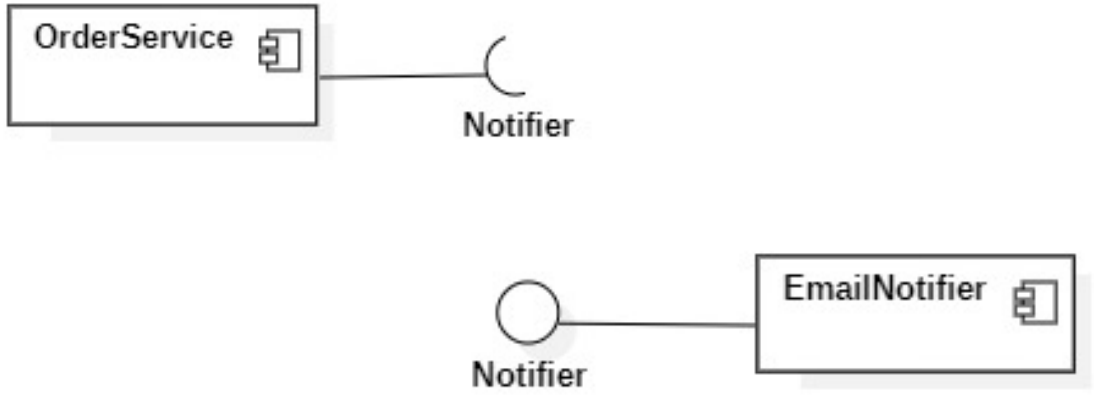
Újrahasznosíthatóság és laza csatolás!

Komponensek összekapcsolása interfészeken

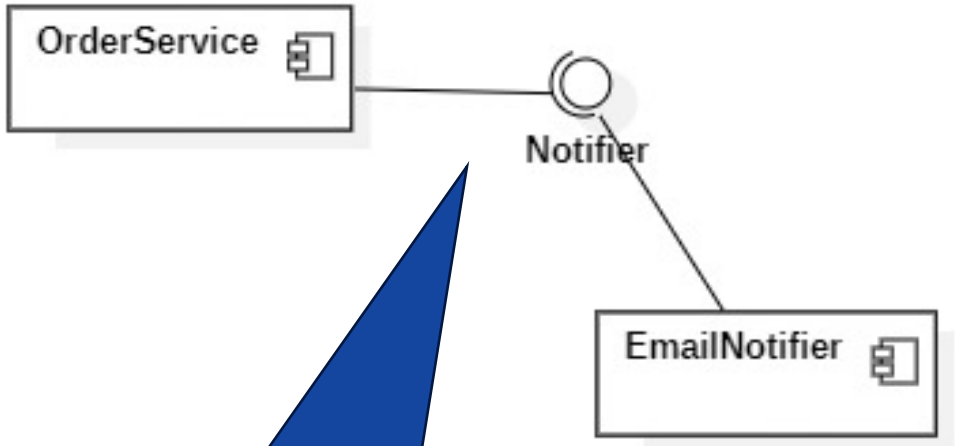
1)



2)

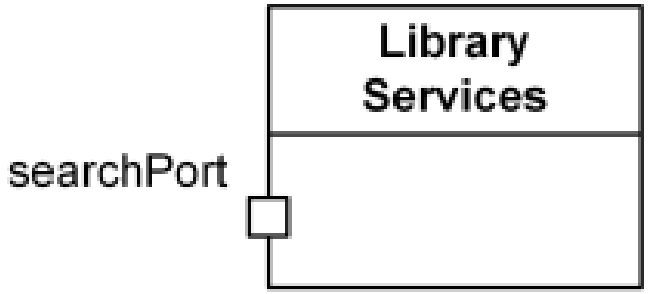


3)

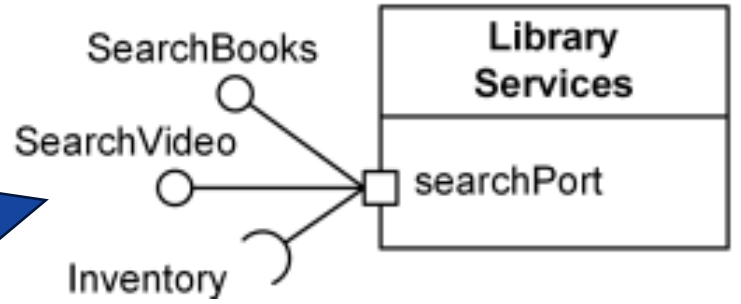


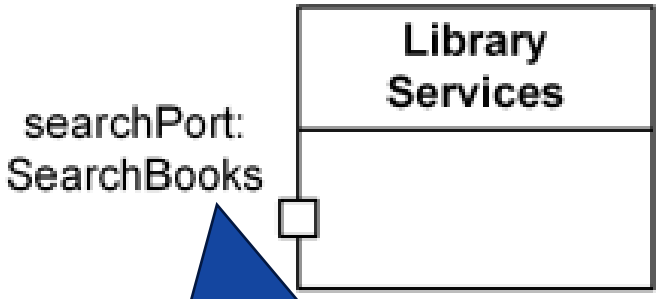
„Ball-and-socket” jelölés, provided és required interfészek összekötése

Port: nevesített kapcsolódási pontok

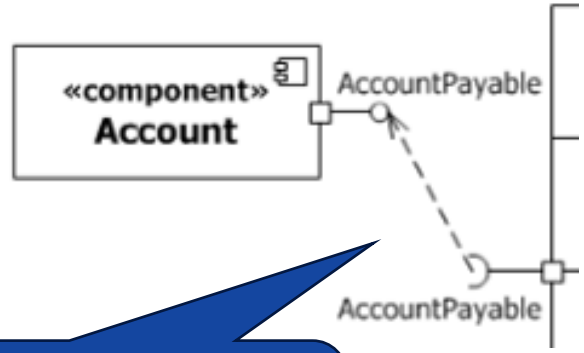


Port interfészekkel (összetett port: több interfésze van)





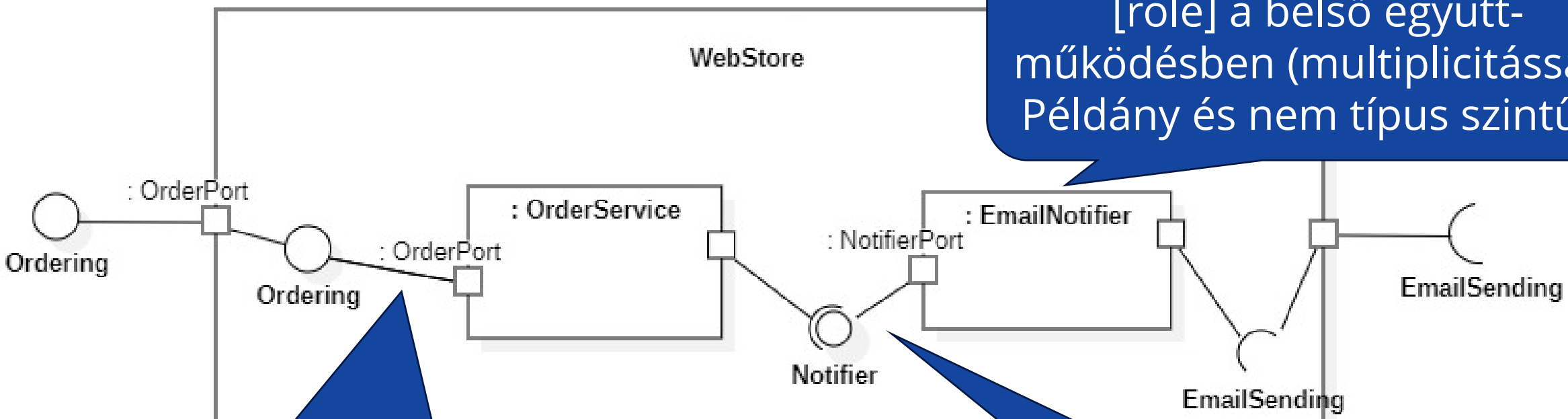
Típusos port



Portok összekötése függőségekkel

Port szerepe: belső felépítés elkülöníthető utána

Belső felépítés (internal structure)



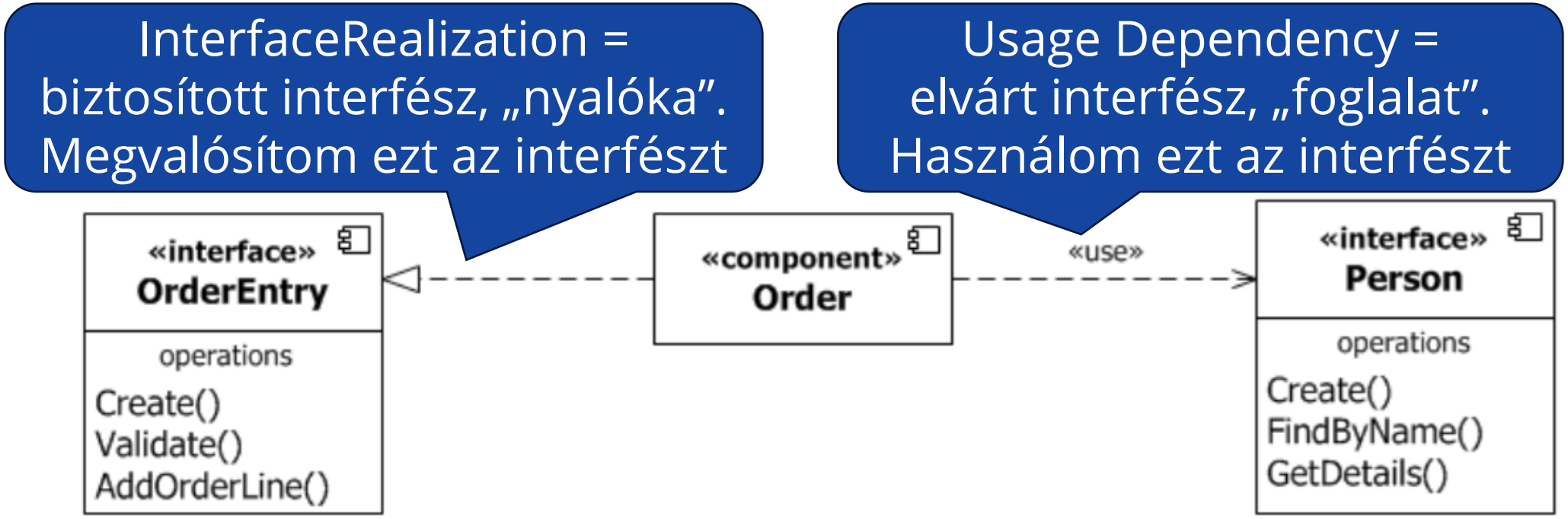
Rész [part]: adott **szerep** [role] a belső együtt-működésben (multiplicitással). Példány és nem típus szintű!!!

Delegációs összeköttetés [delegation Connector]: kérések továbbítása

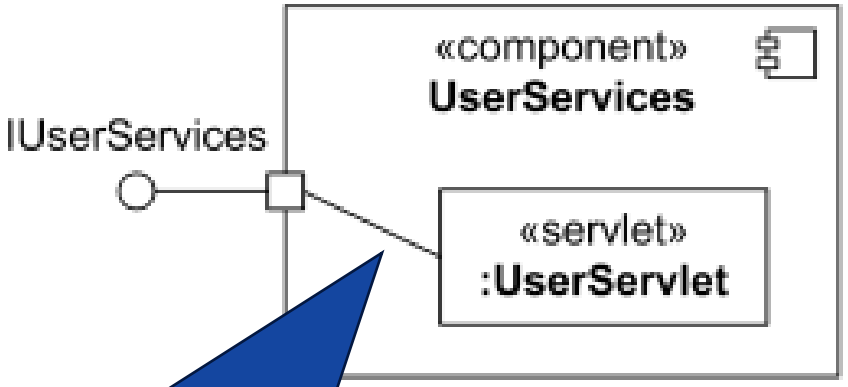
Assembly összeköttetés [assembly Connector]: rendszer összeállítása

Alternatív és további jelölések (1)

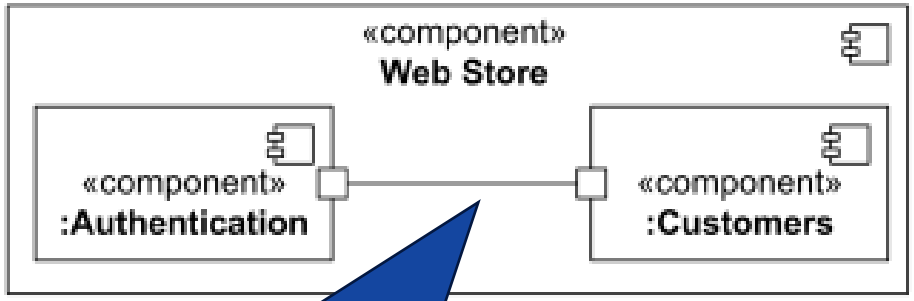
Interfész:



Alternatív és további jelölések (2)



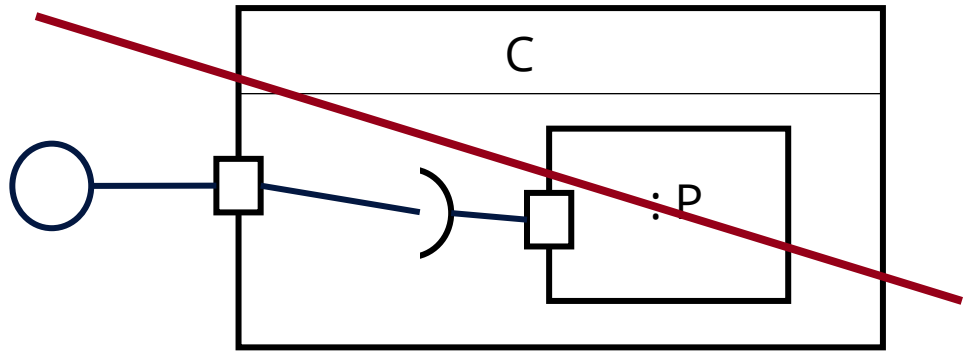
Delegation connector, ha a port összetett és nem egyszerű



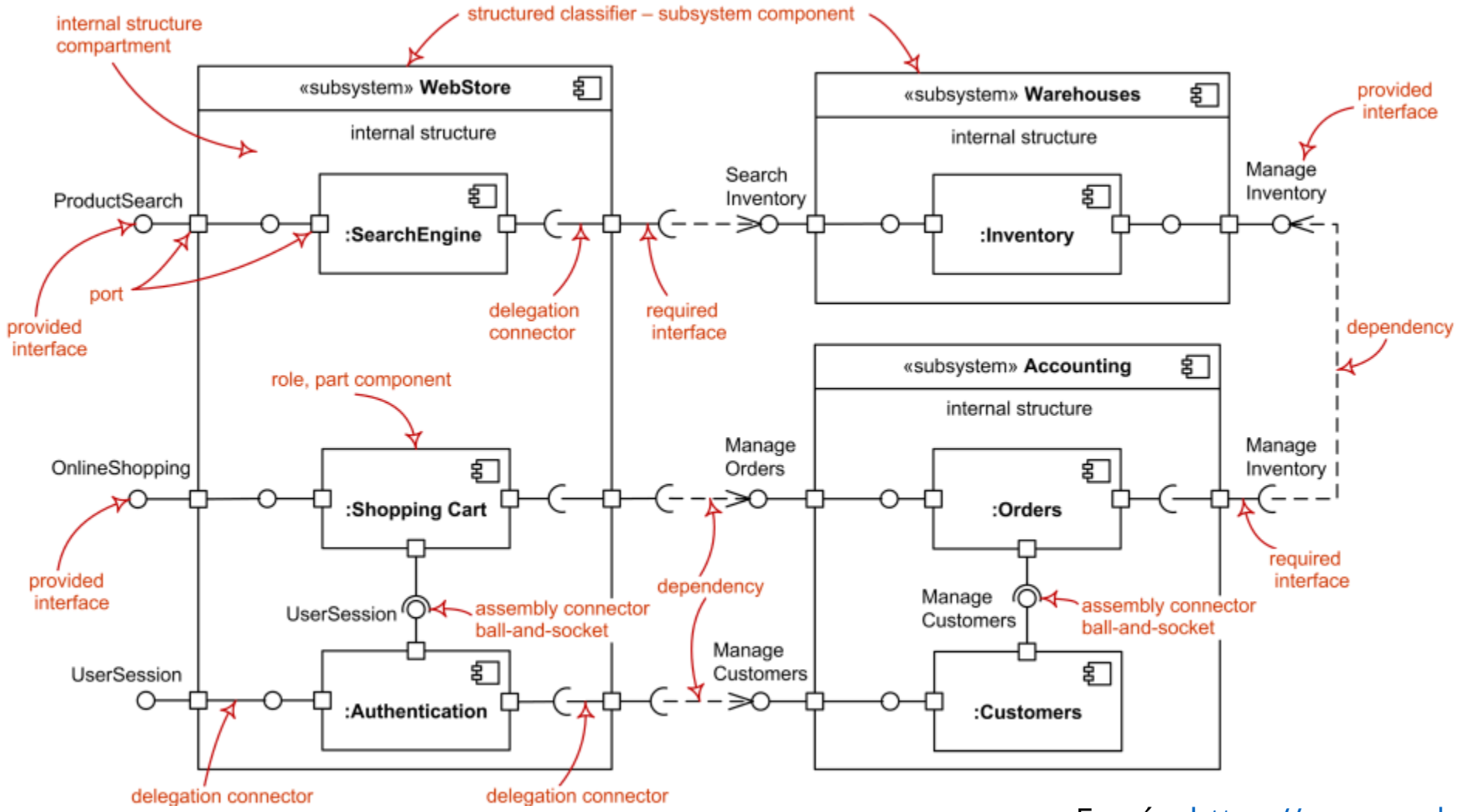
Assembly connector, ha a port összetett (nem egyszerű)

Tipikus hibák

- Delegation esetén ne „forduljon” meg a delegálás iránya
  - Pl., ha a külső komponens megvalósítja az interfészt, akkor a belső rész is



Komponensek összefoglalása



Forrás: https://www.uml-diagrams.org/

FELADAT: HF Portál architektúra



A HF Portál a webes felületet megjelenítő kliensből, és a funkciókat megvalósító szerverből áll.

A kliens a kurzus hallgatóinak állapotát, valamint a beállításokat megjelenítő komponensből áll. A kurzus komponens a fázisok és feladatok részleteit megjelenítő nézeteket tartalmazza.

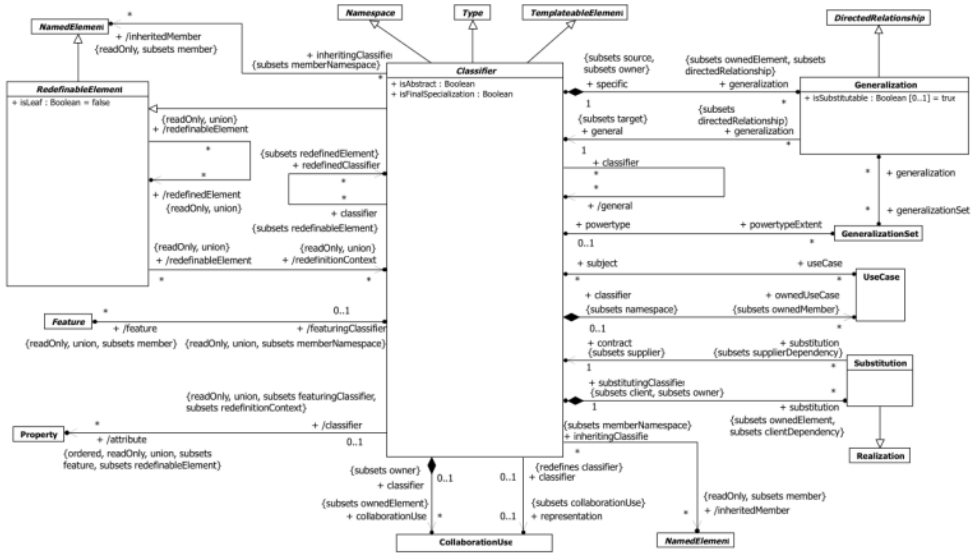
A szerveroldali alkalmazás az adatokat egy adatbázisban tárolja.

Az alkalmazás az ellenőrzések végrehajtásához a hallgatói tárhelyeket helyileg letölti, és ott futtat rajta programokat.

A GitHub-hoz a publikus REST API-n keresztül, az Issues és Pull Request részeket használva csatlakozik.

UML osztály modellek

Osztály, interfész,
attribútum, metódus…



UML osztályokkal nem csak C++/ Java/C# osztályokat lehet modellezni

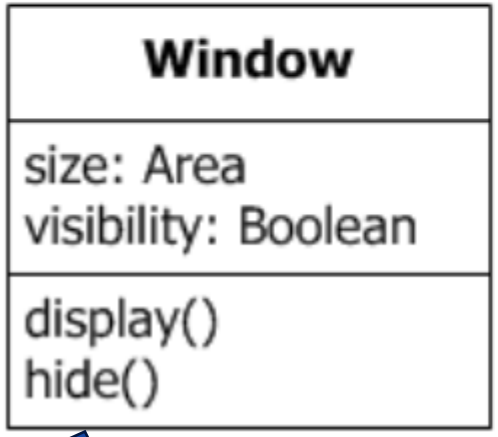
Simple & Structured Classifiers

- „osztálydiagram”
- Leggyakrabban használt UML elemek
- Fogalmak (+tulajdonságaik, műveleteik) és kapcsolatok közöttük
- Változatos absztrakciós szinten használhatók
  - Fogalmi modell, analízis, részletes implementáció…

Részletességi szintek

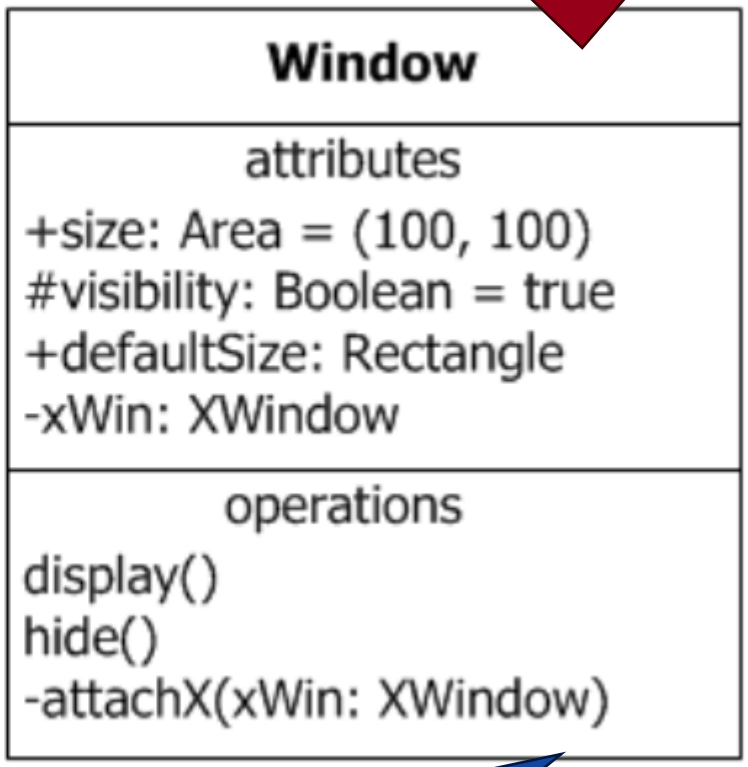
Fogalmak





Kezdeti analízis

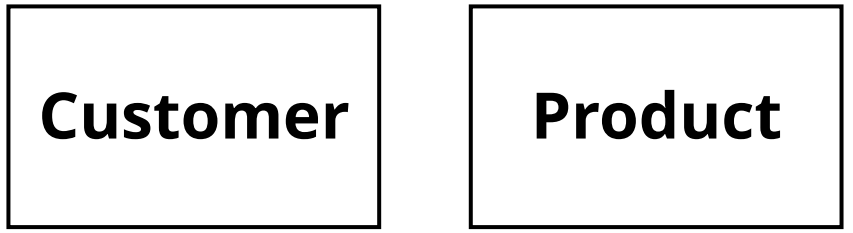
Sokszor nem ezt a szintet kell modellezni!



Részletes implementáció

Osztály [Class]

- *„A modellezett rendszer egy fogalma"*

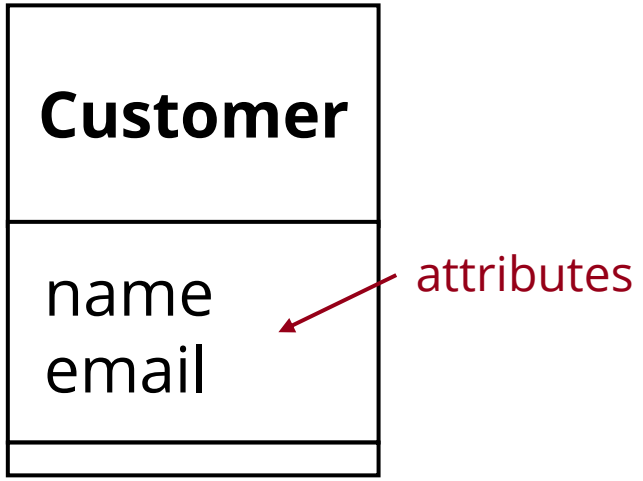


- Dolgok (~objektumok) közös állapota, viselkedése és kényszerei

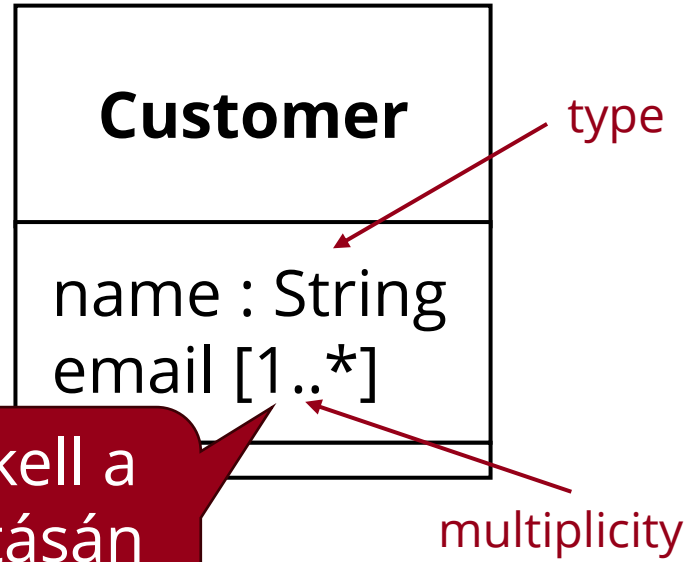
- Elnevezés: főnév egyes számban (domain fogalma)
  - Kerüljük a rövidítéseket!
  - UML megengedi a szóköz használatát (analízis modellben javasolt csak)

Tulajdonság [Property] (attribute)

- Property: példányokhoz értékeket [value] rendel
  - „Futási időben” változhat ez az érték
- Classifier esetén: a példányok „állapotváltozói”
  - Classifier esetén neve: attribute



- Nevén kívül lehet még opcionálisan:
  - Típusa (beépített típus, másik Classifier…)
  - Multiplicitása [alsó határ..felső határ]
    - Hány érték tartozhat hozzá
    - * jelentése: korlátlan
    - Tipikusan: 1, * (0..*), 0..1, 1..*



Még nem kell a megvalósításán gondolkozni!

UML beépített típusok (PrimitiveType)



Figure 21.1 Primitive Types

- Általános, megvalósítástól független típusok
- Matematikai egész, természetes és valós számok
  - Végtelen számosság
- Boolean: *true* és *false* értékek

Abstract Classifier

- Nem lehet példánya
  - Tipikusan a leszármazottainak lesznek

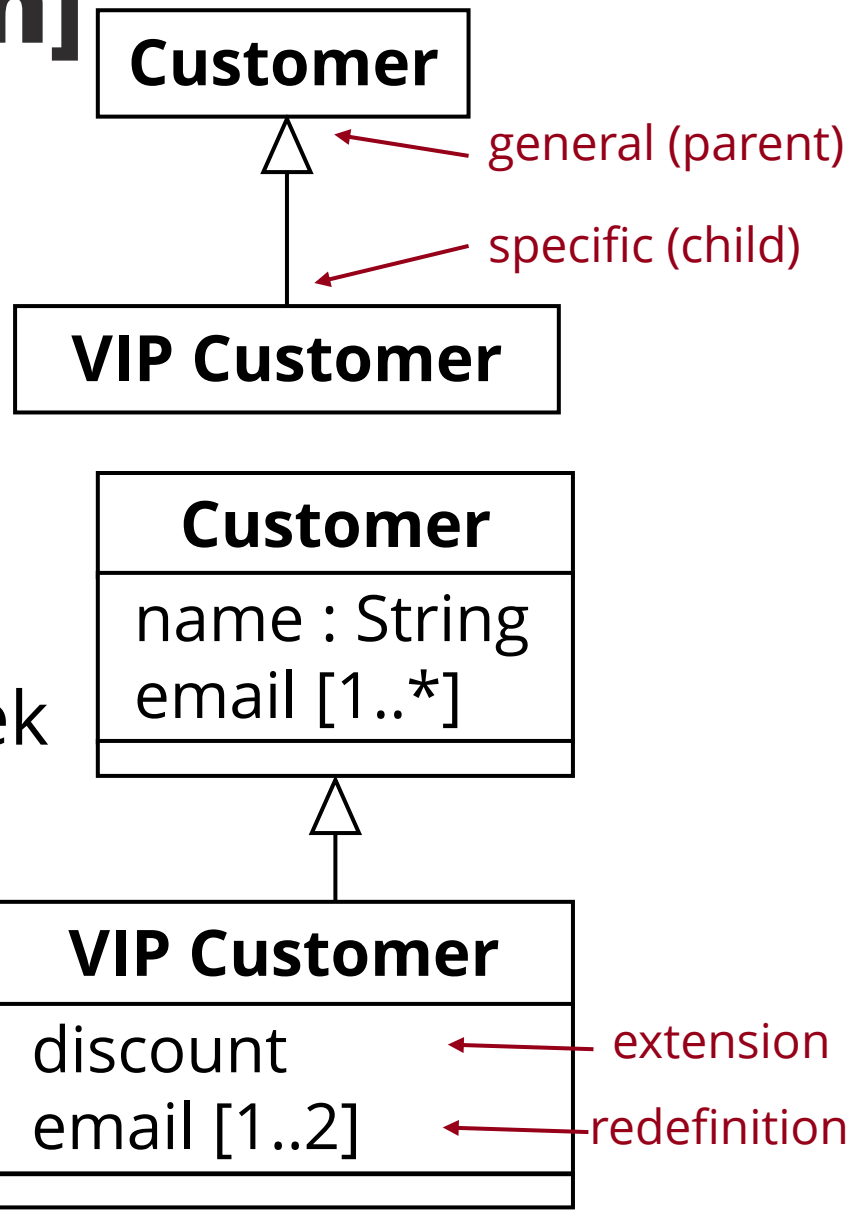
- Jelölés:



- (Nem csak osztály lehet absztrakt, lehet viselkedés is…)

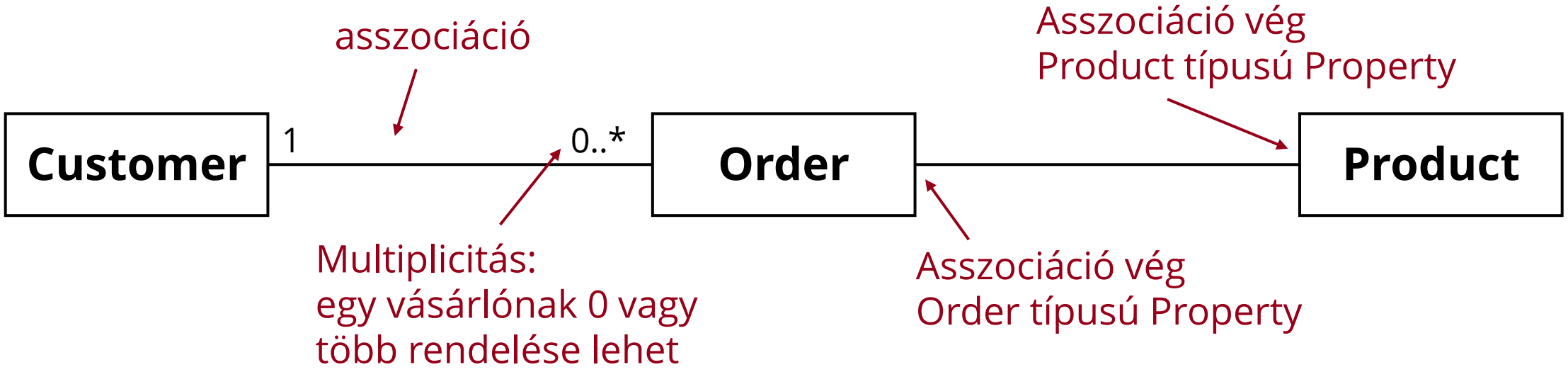
Általánosítás [Generalization]

- Általánosítás/specializálás kapcsolat
  - Megörökli [inherit] a szülő tagjait [member]
    - Átláthatóbb, karbantarthatóbb a modell
  - Helyettesíthetőség [subsitutability]
    - Nem minden esetben!
- Több általánosítása is lehet egy classifier-nek
- Kibővítheti vagy felüldefiniálhatja a szülőt
  - struktúrát és viselkedést is
  - *Redefinition*: nem ellentmondó módon



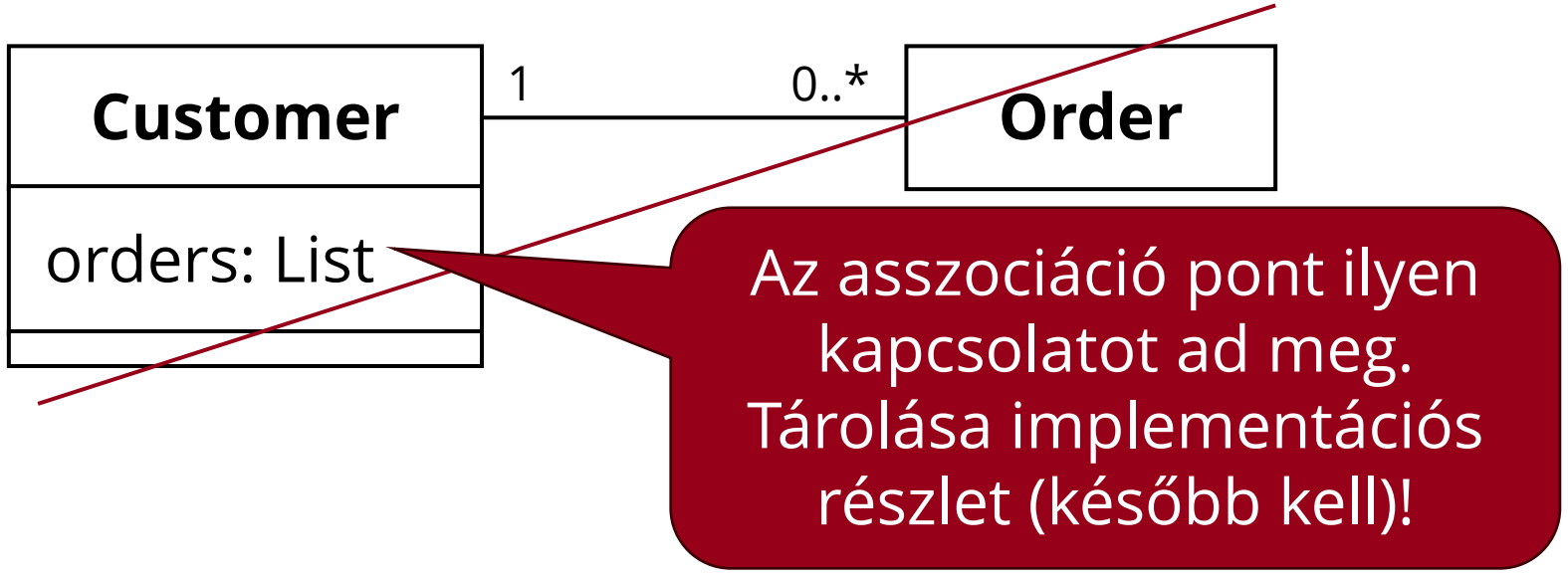
Asszociáció [Association]

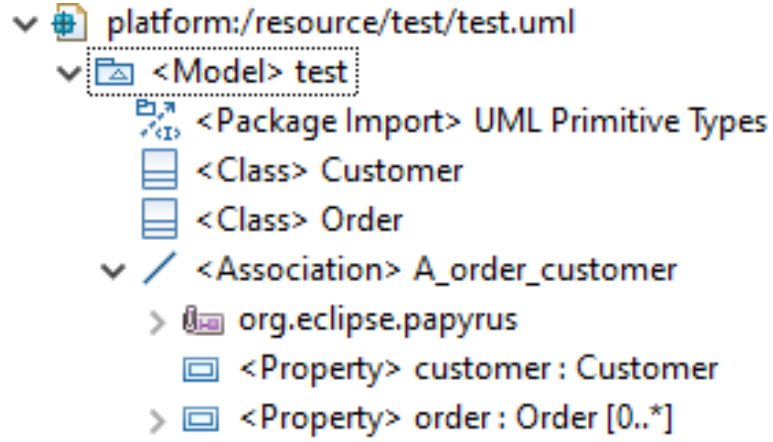
- Szemantikai kapcsolat típussal rendelkező példányok között
- „Permanens” kapcsolat (nem csak egy metódushívás idejéig)



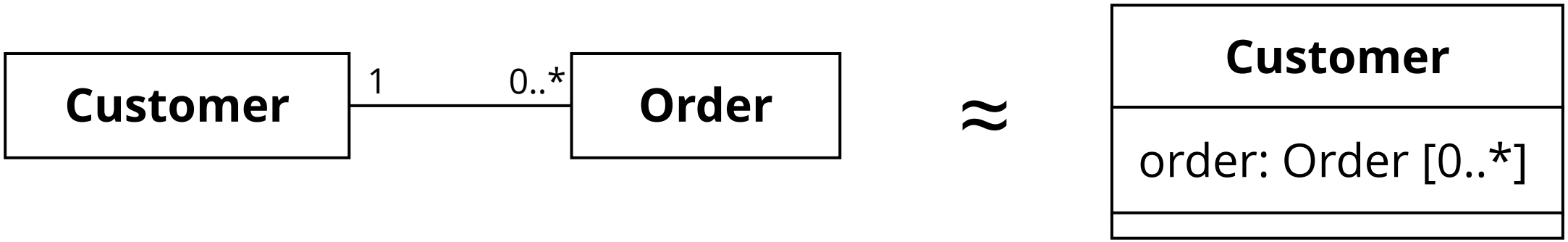
- Asszociáció vég [association end]: ez is egy Property
  - Alapesetben az asszociáció tárolja ezeket a Property-ket.

Asszociációk és attribútumok





FIGYELEM: attribútum és asszociáció hasonló fogalmakat modellez, csak mást hangsúlyoz, máshogy bővíthető

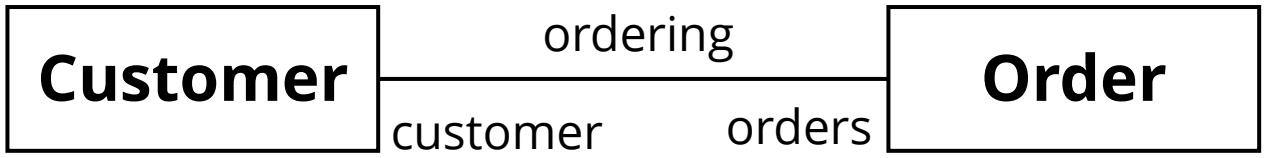


Nem kell az asszociációhoz külön „tároló” attribútumot felvenni.

Asszociációk elnevezése

- Asszociáció és a végek is külön-külön elnevezhetők
  - Alapesetben a végek neve a hozzá tartozó classifier kis betűvel
- Akkor érdemes, ha nem egyértelműen kitalálható

Feleslegesen kiírt nevek



Jó példa: név plusz információt hordoz

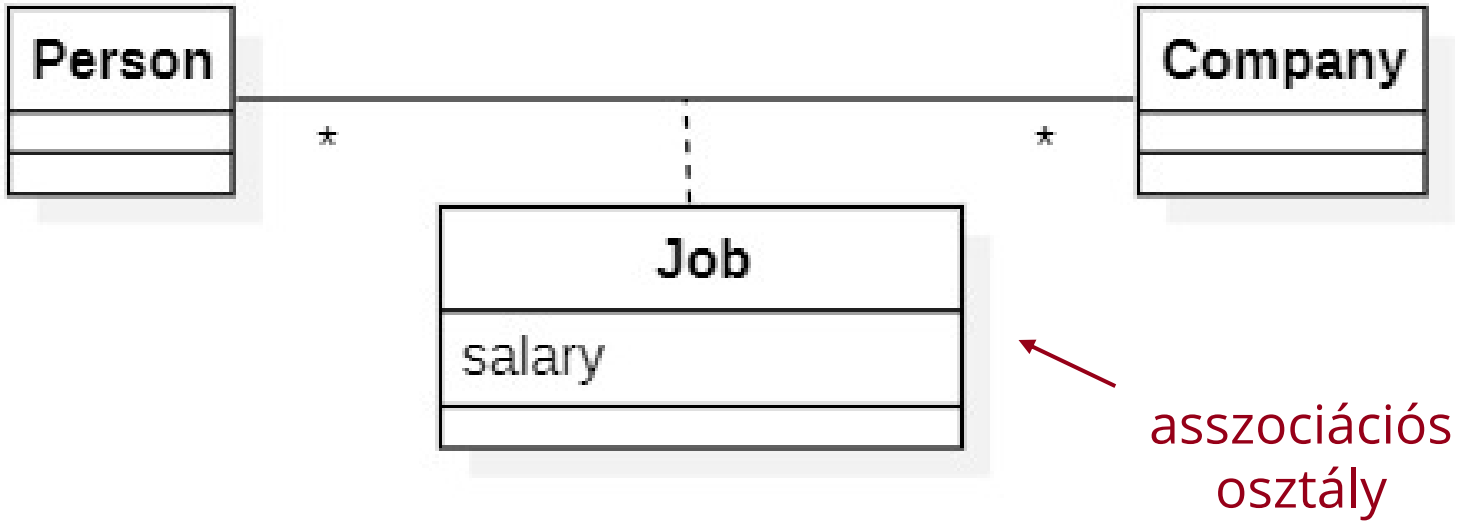


Forrás: uml-diagrams.org

- Különösen fontos, ha ugyanazok között több asszociáció van

Asszociációs osztály [Association class]

- Bizonyos információ a kapcsolathoz tartozik
- Érdemes használni a kezdeti modellezés során
  - Később megvalósítható többféleképpen



FELADAT: HF portál fogalmi modell (1)



A HF portálon lehet kurzusokat létrehozni, amiknek nevük és típusuk van. A felhasználókhoz tároljuk a vezeték- és keresztnevüket, az egyedi Moodle-azonosítójukat, valamint a nyelvüket (magyar, német, angol). A felhasználók kurzusokhoz tartozhatnak. Tároljuk a felhasználó kurzusban betöltött szerepét. Szerep a hallgató, az oktató és a tárgyfelelős.

Egy kurzushoz fázisok tartozhatnak, amik a házi feladat határidővel rendelkező szakaszait adják meg. A fázisok feladatokból állnak, amiknek szintén van saját határidejük. Minden fázisnak van egy neve és határideje.

Példányok: InstanceSpecification

NINCS olyan statikus modellelem, hogy Objektum [Object]

- Példányokat lehet modellezni: InstanceSpecification
  - Példák lehetséges vagy létező példányokra
  - Részleges, absztrakt leírások általában
- Példány nevének és típusainak megadása opcionális



Példánymodellre vonatkozó kényszerek

Mire figyeljünk egy példánymodell létrehozásakor?

- Megfelel a példányok típusai által megfogalmazott kényszereknek
- Csak olyan slot-jai vannak a példánynak, ami a típusban definiált
- Csak olyan példányok között van Link, amiknek a Classifier-jei között van asszociáció
- Az asszociációk multiplicitási kényszerei be vannak tartva a példánymodellben (minimum és maximum számosság)

FELADAT: HF portál fogalmi modell (2)



- Rajzoljunk példánymodell(ek)et, hogy megértsük a modellt!

- Például:
  - az Onboarding fázis esetén Onboarding feladattal Figy Elek hallgatja a Szoftvertechnológia kurzust.
  - A regisztrációval (GitHub, SonarQube) már meg van, jelenleg az Onboarding feladata vár kiértékelésre…

- Lett újabb kérdésünk a példányok modellezése kapcsán?

FELADAT: HF portál fogalmi modell (3)



A HF portálon lehet kurzusokat létrehozni, amiknek nevük és típusuk van. A felhasználókhoz tároljuk a vezeték- és keresztnevüket, az egyedi Moodle-azonosítójukat, valamint a nyelvüket (magyar, német, angol). A felhasználók kurzusokhoz tartozhatnak. Tároljuk a felhasználó a kurzusban betöltött szerepét. Szerep a hallgató, az oktató és a tárgyfelelős.

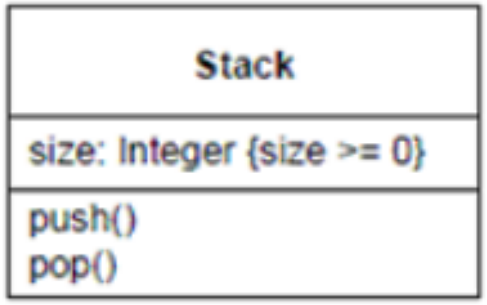
Egy kurzushoz fázisok tartozhatnak, amik a házi feladat határidővel rendelkező szakaszait adják meg. Minden fázisnak van egy neve és határideje. A fázisok feladatokból állnak, amiknek szintén van saját határidejük.

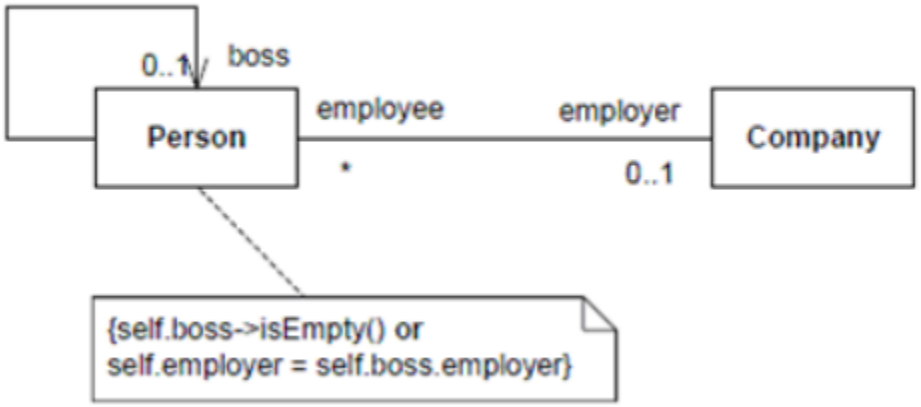
Egy kurzust hallgató szerepben végző felhasználó számára minden kurzushoz tartozó fázishoz, valamint fázishoz tartozó feladathoz tároljuk, hogy az adott fázisnak / feladatnak mi az állapota (elkezdte, dolgozik, kiértékelésre vár, kész), valamint hogy azt a felhasználó mikor kezdte el, teljesítette sikeresen.

Modellezés egyik fontos célja, hogy segítsen kérdéseket megfogalmazni

Kényszer [Constraint]

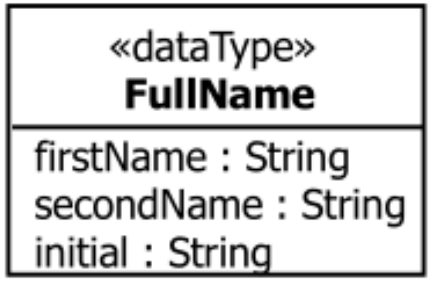
- További kényszer, amit az érvényes modellnek teljesítenie kell
- Sok féle elemhez rendelhető (osztály, attribútum, művelet…)
- Jelölés: { } között vagy külön jegyzetben

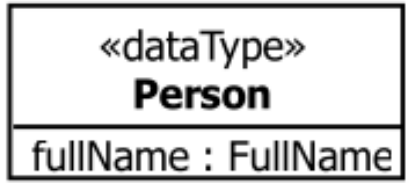


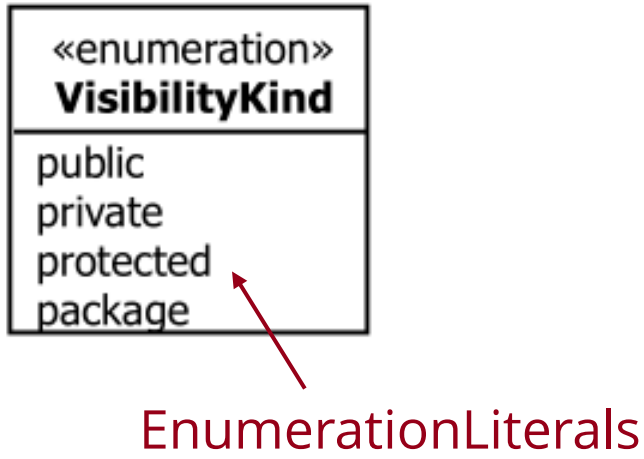


DataType, Enumeration

- DataType: Példányaik csak értékükben különböznek egymástól

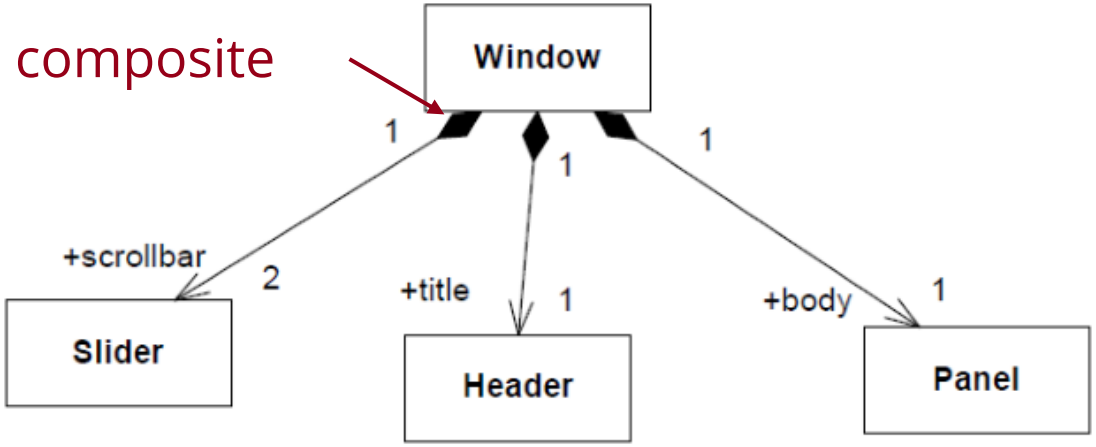






Aggregáció (Aggregation)

- Shared aggregation: példányok csoportosítása
  - Egyvalaki több csoportosításban is részt vehet
- Composite aggreation (composition): rész/egész kapcsolat
  - Erősebb forma: egyvalaki csak egy kompozícióban vehet részt
  - Ha a tartalmazó példányt töröljük, a rész példányok is törlődnek
- Csak bináris asszociáció, csak az egyik vége lehet, nem lehet kör

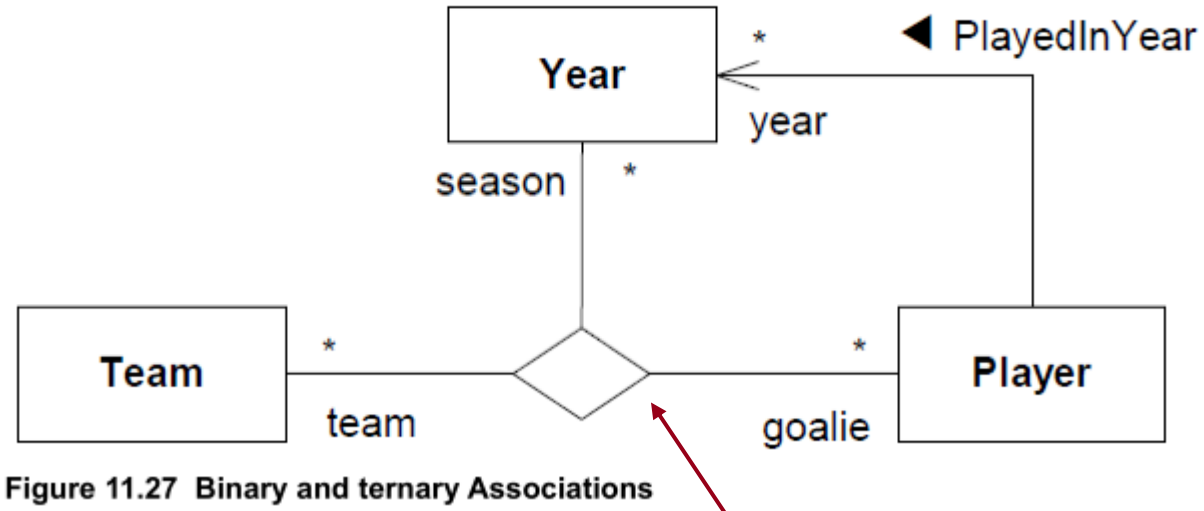




„The distinction between aggregation and association is often a matter of taste”

Többes asszociáció

Nem csak bináris asszociációt lehet modellezni



Asszociáció navigálhatósága

- Példányok „hatékonyan” elérhetők a másik oldali példányokból
  - Hatékonyság jelentése implementációfüggő (pl. referencia van rá)

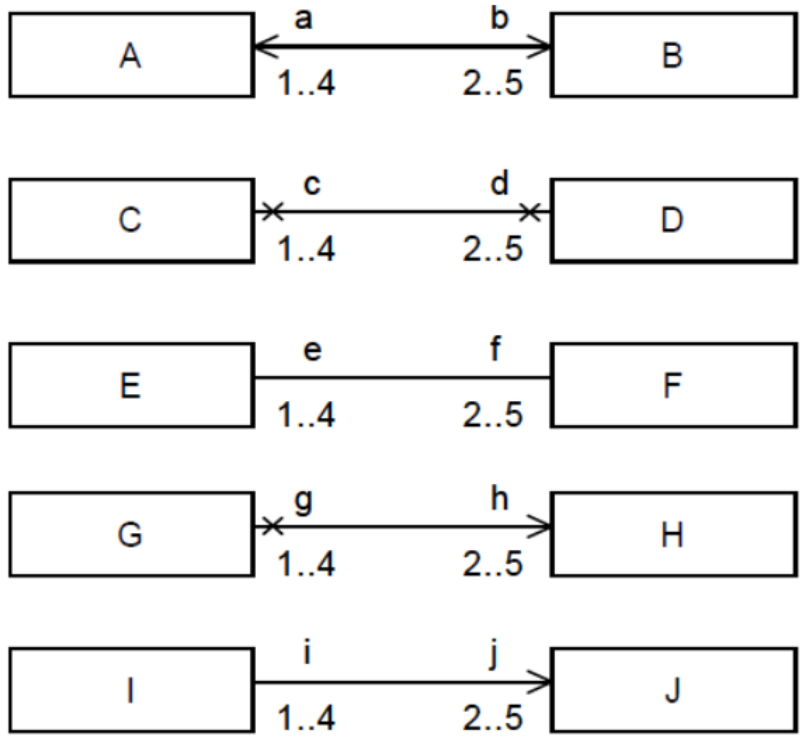


Figure 11.29 Examples of navigable association-owned ends

A, B: Navigálható mindkét irány

C, D: Egyik irány sem navigálható

E, F: Nem specifikált. (De lehet, hogy navigálható.)

G példányai elérik a hozzá tartozó H példányokat. H példányai nem érik el könnyen a megfelelő G példányokat.

Hasonló a G, H példához, csak itt a diagramon nem jelenítjük meg a kereszt jeleket az olvashatóság kedvéért.

Asszociációvég birtoklása [owned end]

- Asszociációvéget reprezentáló tulajdonságot birtokolhatja:
  - maga az asszociáció vagy
  - a másik végen lévő classifier (jelölés: pont [dot] a vonal végén)
- (Nem minden eszköz támogatja, ritkábban használják)

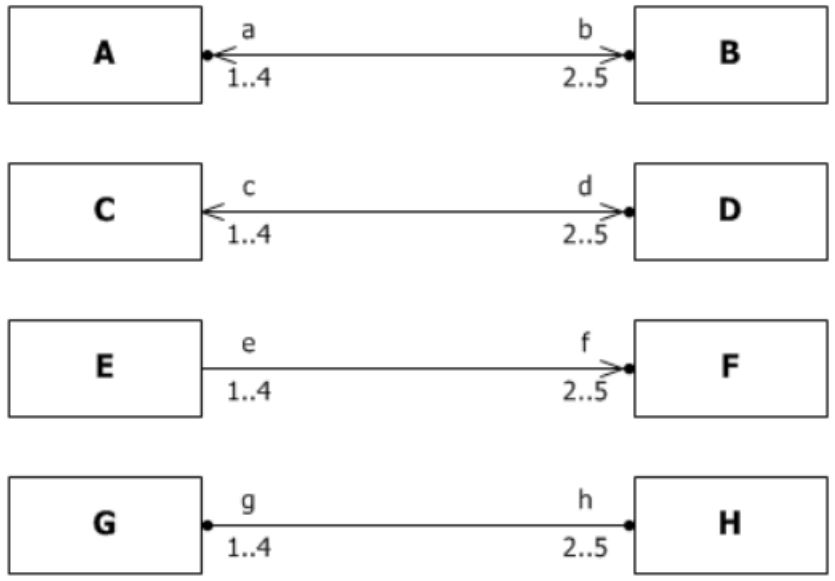


Figure 11.30 Examples of class-owned ends

A osztály birtokolja **b** assz. véget
B osztály birtokolja **a** assz. véget

C osztály birtokolja **d** assz. véget
asszociáció birtokolja **c** assz. véget

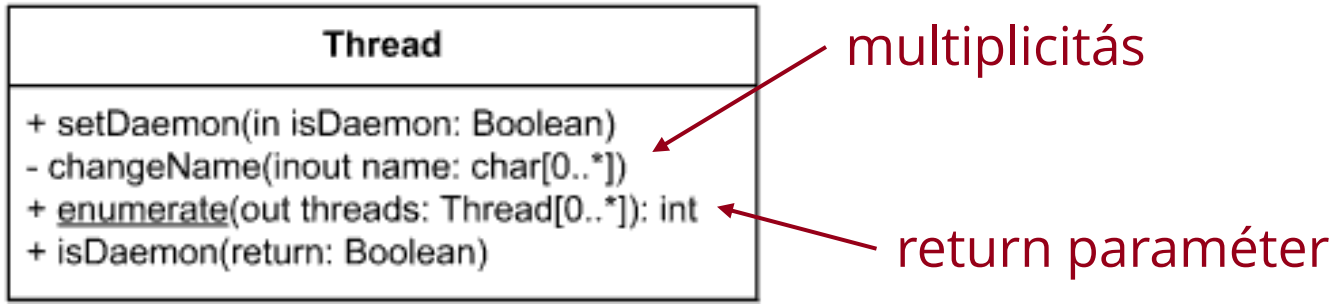
E osztály birtokolja **f** assz. véget
asszociáció birtokolja **e** assz. véget

G osztály birtokolja **h** assz. véget
H osztály birtokolja **g** assz. Véget
(mindkét irány navigálható, mert osztály birtokolja)

Viselkedés definiálása

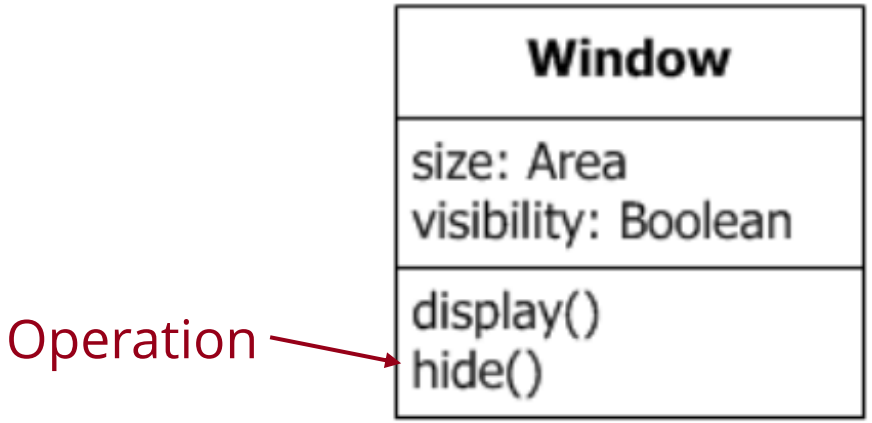
BehavioralFeature, Parameter

- Behavior: specifikáció, hogy hogyan változik az állapot az időben
- BehavioralFeature: reagál a Feature meghívására [invocation]
  - Viselkedés megadása: szöveg, állapotgép, aktivitás…
- Lehet statikus (jelölés: aláhúzás) és/vagy absztrakt is
- Parameter: lehetnek paraméterei
  - Irány: in, inout, out, return
  - Egyes paraméternek van számossága (0 alsó határ: opcionális)



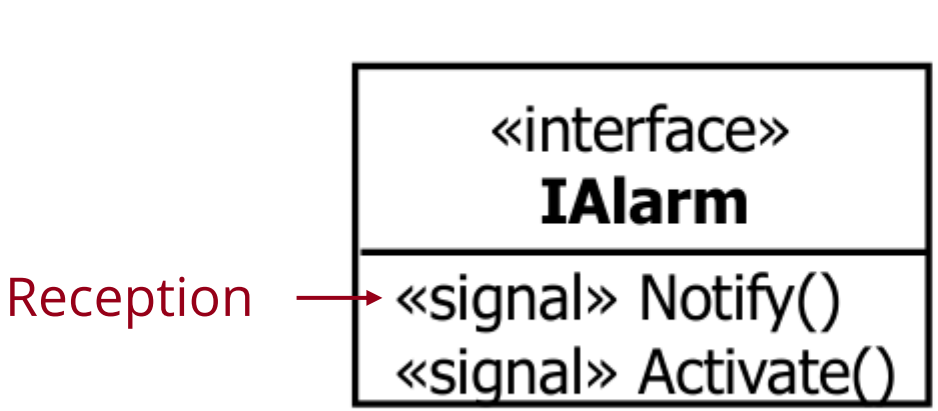
Művelet [Operation]

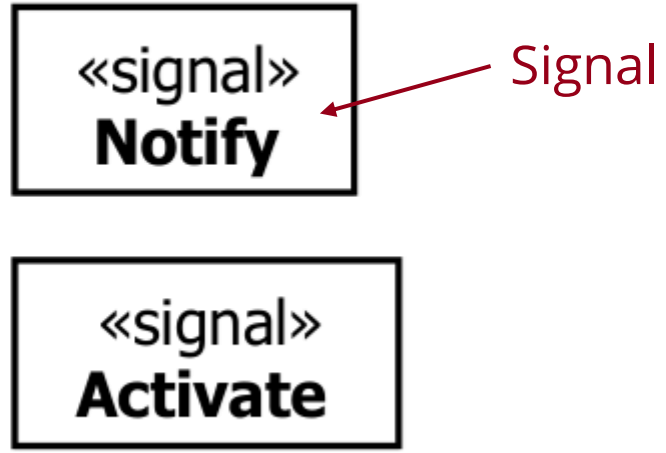
- Művelet [Operation]: Class, DataType vagy Interface viselkedése
- Szinkron vagy aszinkron módon is meghívható
- Metódus [method]: a BehavioralFeature-t megvalósító viselkedés
  - UML támogatja a polimorfizmus modellezését
  - Meghívandó metódus lehet a leszármazottban lévő is



Signal, Reception

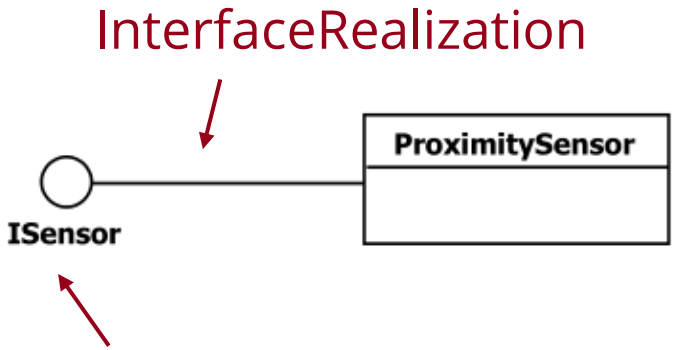
- Aszinkron kommunikáció modellezésére
  - A hívó fél nem várja meg a választ (nem blokkol)
- Jelzés [Signal]: aszinkron kommunikációs mód specifikációja
  - Lehetnek attribútumai is
- Fogadás [Reception]: adott típusú Signal-t tud fogadni az osztály

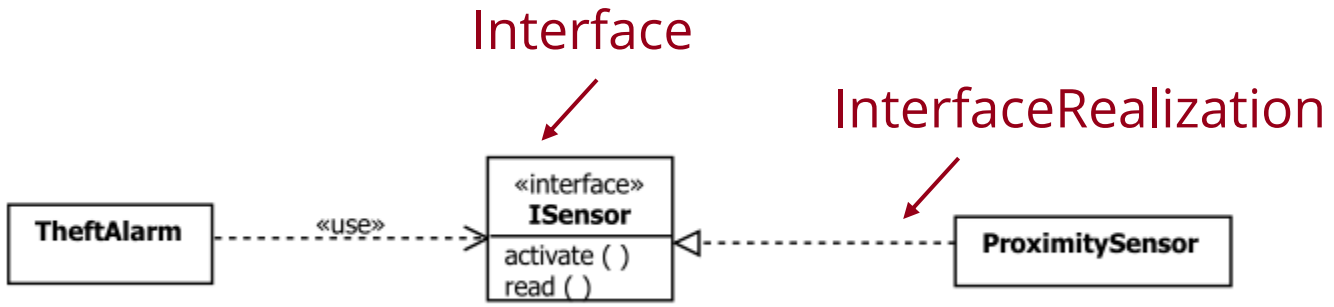


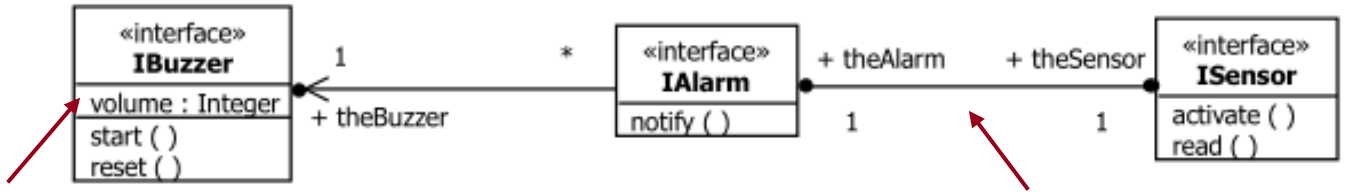


Interfész [Interface]

- Nyilvános, összetartozó „szerződés”
- Komponenseknél már látott jelölés







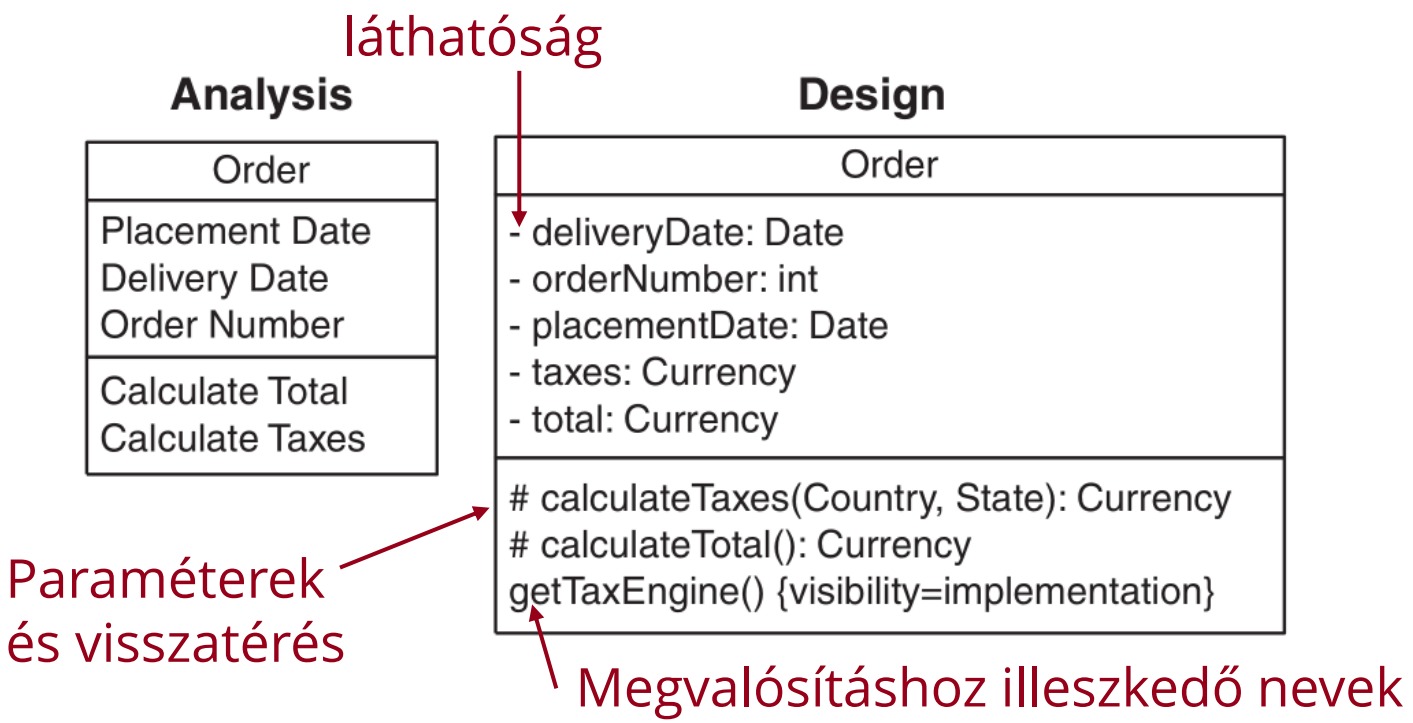
Attribútum interfészen (óvatosan!)

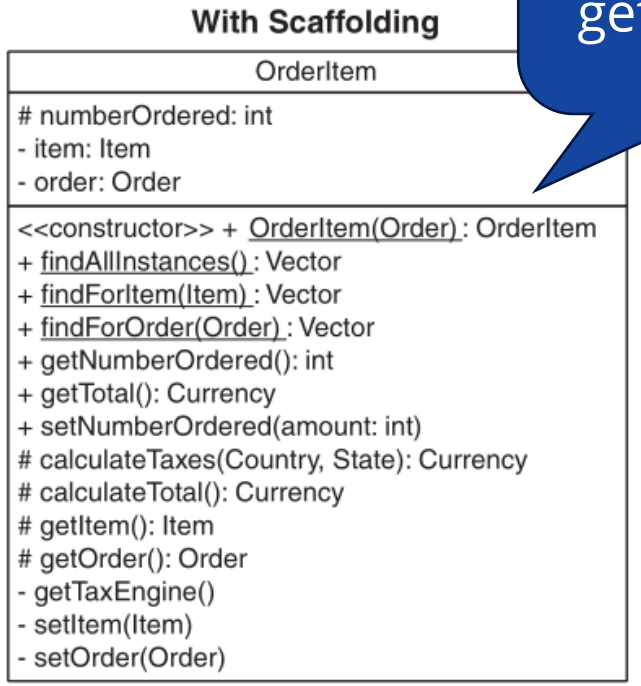
Asszociáció interfészek között

Implementációs nézet és lehetséges megvalósítások

Analízis és részletes nézetek

- Tervezés során egyre több részletet adhatunk a modellhez
- Melyik szinten érdemes megállni?





Ez a szint már ritkán kell (pl. getter/setter)

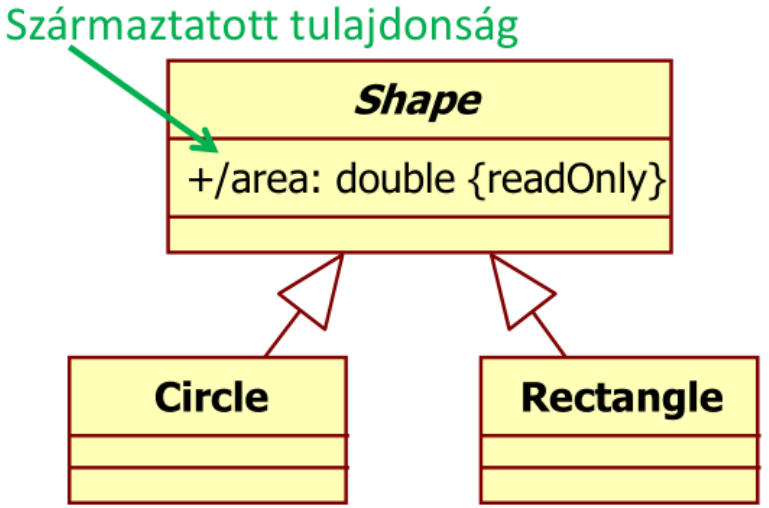
Láthatóság

- Műveletekhez és tulajdonságokhoz megadható
- Láthatóság fajtái:
  - **private** (**-**): csak az adott osztály tagjai számára látható
  - **protected** (**#**): csak az adott osztály és a leszármazottak tagjai számára elérhető
  - **public** (**+**): bárki számára elérhető, aki az osztályt eléri
  - **package** (**~**): bárki számára elérhető, aki ugyanabban a csomagban van, mint az osztály

- FIGYELEM: eltérhet a jelentése egy-egy programozási nyelvétől

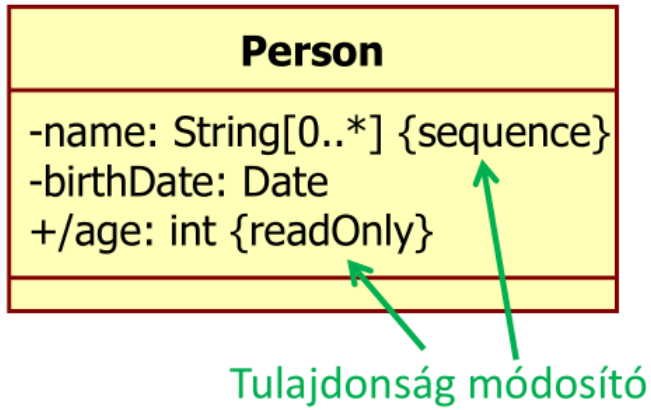
Származtatott tulajdonság [derived property]

- Tulajdonság értéke valami számítás eredménye
- Számítást nem feltétlen adjuk még meg
- Jelölés: a név előtti perjel (/)



Tulajdonság módosító [property modifier]

- A tulajdonság jelentését pontosítják
- Jelölés: Kapcsos zárójelben a tulajdonság után



Módosító	Jelentés
readOnly	a tulajdonság csak olvasható
union	a tulajdonság értéke a részhalmazaiból számolt unió
subsets <*propname*>	a tulajdonság a <*propname*> részhalmaza
redefines <*propname*>	a tulajdonság átdefiniálja az örökölt <*propname*> tulajdonságot
ordered	az értékek sorrendjét megtartja
unordered	az értékek sorrendjét nem feltétlenül tartja (ez az alapértelmezett)
unique	a több értékű tulajdonságban nincsenek duplikáltan tárolt értékek
nonunique	a több értékű tulajdonságban lehetnek duplikáltan tárolt értékek
sequence *(or seq)*	a tulajdonság egy lista (nonunique és ordered)
id	a tulajdonság részt vesz az objektumok azonosításában

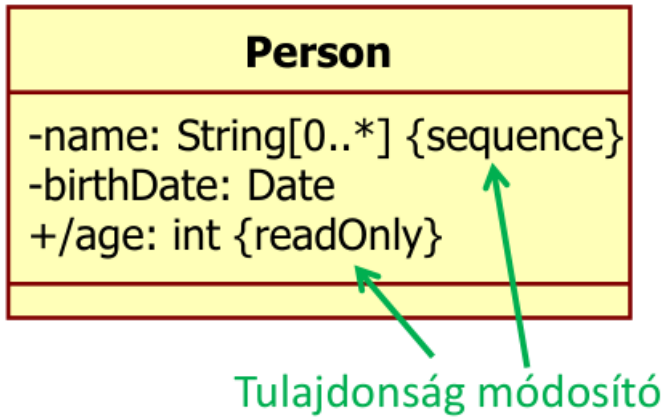
Használhatók asszociáció végeken is!

Hogyan valósítanánk meg? (példák)



Java leképzés:

```
public class Maze {
  private ArrayList<Field> fields;
}
public class Field {
  private Maze maze;
}
```



Java leképzés:

```
public class Person {
    private List<String> name;
    private DateTime birthDate;

    public List<String> getName() { return name; }
    public DateTime getBirthDate() { return birthDate; }
    public void setBirthDate(DateTime value) { birthDate = value; }
    public int getAge() { /*...*/ }
```

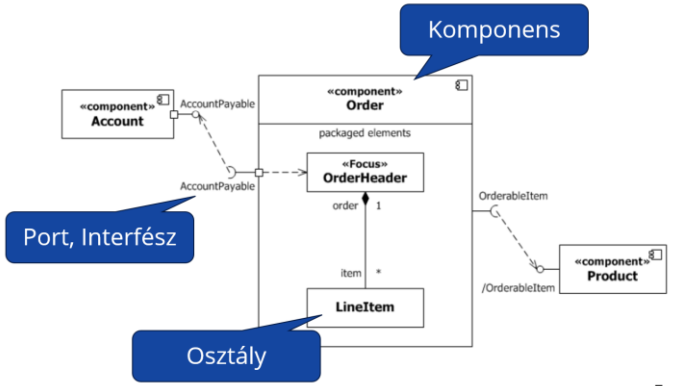
További elemek és felhasználásuk

- Beágyazott osztályok [nested class]
- Minősítő [qualifier]: minősített asszociációvég partíciókra osztja a másik oldalon lévő példányokat
- Csomag [package]: többi elem hierarchikus rendszerezése
  - Programozási nyelvekben: `namespace`, `package`…

- Generalization használható asszociációk között is
- Függőség [dependency] használható osztályok között is

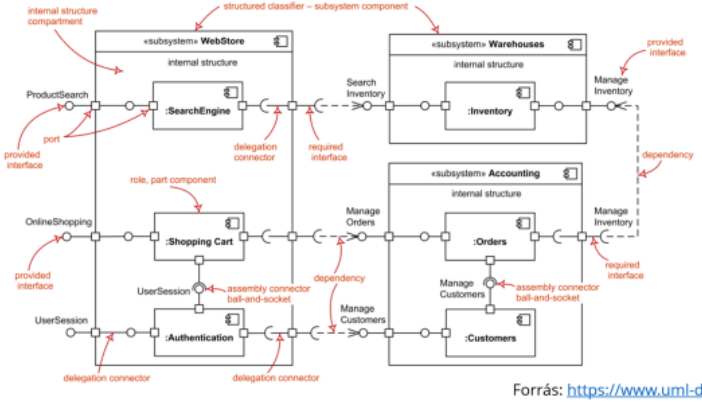
Összefoglalás

UML struktúra modellek



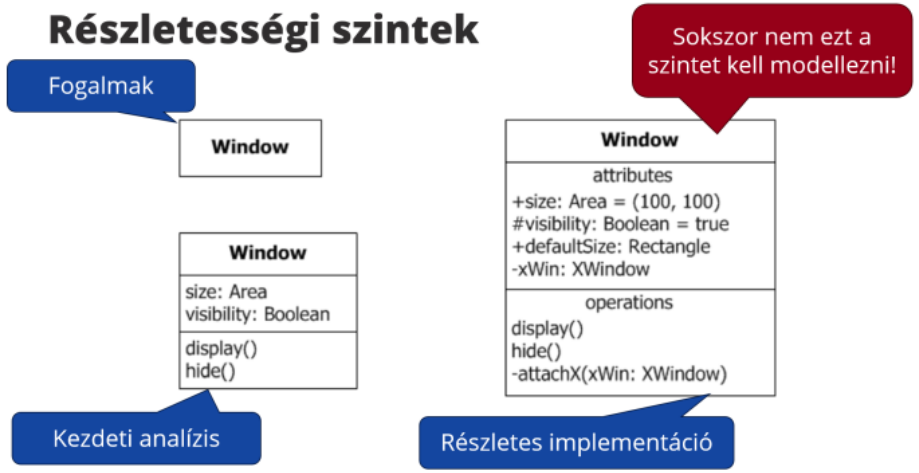
Forrás: UML v2.5.1

Komponensek összefoglalása



Forrás: https://www.uml-diagrams.org/

Részletességi szintek



Példa: összetett UML osztály-diagram

Össze tudjuk foglalni 3-4 mondatban, hogy milyen szakterületet ír le és mik a főbb felelősségek?

